

DISEÑO DE ARQUITECTURA DE SOLUCIONES

Sistema de Banca por Internet BP

Presentado por:

Ing. Diego Cuaycal

Perfil:

Ingeniero de Software

Tipo de Evaluación:

Prueba Técnica - Arquitectura



github.com/DiegoCuaycal



dev-portfolio-diego.vercel.app/

Contenido

1. RESUMEN EJECUTIVO	2
2. DECISIONES ARQUITECTÓNICAS Y STACK TECNOLÓGICO.....	2
2.1. Aplicación Móvil	2
2.2. Frontend Web	3
2.3. Backend.....	4
2.4. Infraestructura en la Nube	4
3. ARQUITECTURA DE AUTENTICACIÓN Y SEGURIDAD	5
3.1. Flujo Óptimo para OAuth 2.0	5
3.2. Integración de Onboarding Biométrico y Accesos Posteriores	6
4. CAPA DE INTEGRACIÓN, MICROSERVICIOS Y PATRONES DE DATOS.....	7
4.1. API Gateway y Expansión de Microservicios	7
4.2. Persistencia para Clientes Frecuentes mediante Cache-Aside	7
4.3. Auditoría Desacoplada con Patrón Publish-Subscribe	8
4.4. Sistema Desacoplado de Notificaciones Omnicanal	8
5. ARQUITECTURA DE SOFTWARE (MODELO C4).....	8
5.1. Nivel 1: Diagrama de Contexto del Sistema	9
5.2. Nivel 2: Diagrama de Contenedores	9
5.3. Nivel 3: Diagrama de Componentes (Microservicio de Transferencias)	10
6. CONSIDERACIONES NO FUNCIONALES Y NORMATIVAS.....	11
6.1. Seguridad y Cumplimiento Bancario	11
6.2. Alta Disponibilidad y Tolerancia a Fallos	12
6.3. Rendimiento y Escalabilidad.....	12

1. RESUMEN EJECUTIVO

Este documento presenta el diseño arquitectónico propuesto para el nuevo Sistema de Banca por Internet de la entidad financiera BP. La solución planteada se basa en una arquitectura de microservicios, alojada en la nube (Cloud-Native) y con un alto nivel de desacoplamiento entre sus componentes.

El objetivo principal es ofrecer a los usuarios una experiencia rápida y sin interrupciones, tanto en la plataforma web como en la aplicación móvil, permitiéndoles consultar su historial y realizar transferencias de forma ágil. Todo esto se logra garantizando el estricto cumplimiento de las normativas bancarias, alta disponibilidad (HA) y tolerancia a fallos. Además, la seguridad es el pilar central de este diseño, por lo que se integra autenticación biométrica y el estándar OAuth 2.0 para proteger cada acceso.

2. DECISIONES ARQUITECTÓNICAS Y STACK TECNOLÓGICO

A continuación, detallo las tecnologías seleccionadas para cada capa del sistema. Cada decisión fue evaluada priorizando los requerimientos críticos de la entidad BP: seguridad integral, procesamiento en tiempo real y escalabilidad en la nube.

2.1. Aplicación Móvil

Para el desarrollo de la capa móvil, el requerimiento exigía una solución multiplataforma. Al someter a evaluación a los líderes del sector, React Native y Flutter, busqué una tecnología que permitiera mantener un único código base para iOS y Android sin sacrificar en lo absoluto la experiencia de usuario (UX) ni la seguridad del cliente. Tras analizar el costo-beneficio de ambos frameworks en un contexto estrictamente bancario, seleccioné *Flutter* como la tecnología base por los siguientes motivos:

- *Rendimiento optimizado para Onboarding Biométrico:* Dado que el requerimiento exige validación por reconocimiento facial, recomiendo integrar servicios cognitivos de industria como Azure Face API para delegar el análisis biométrico con alta precisión y detección de vida. Para que esta integración sea

exitosa, la aplicación debe procesar el flujo de la cámara en tiempo real sin cuellos de botella. Flutter compila directamente a código nativo (AOT) mediante Dart, prescindiendo de puentes de comunicación intermedios, lo que garantiza una ejecución fluida y latencia casi nula

- *Protección proactiva contra Ingeniería Inversa:* Al tratar con transacciones financieras, el lado del cliente es un vector de ataque crítico. El código ya compilado de Flutter es inherentemente más complejo de desensamblar en comparación con frameworks basados puramente en JavaScript. Esto añade una barrera de seguridad fundamental para mitigar riesgos de clonación o manipulación maliciosa de la app.

2.2. Frontend Web

El canal web transaccional es la cara principal del banco frente al usuario, por lo que no podemos darnos el lujo de armar una arquitectura basada en librerías fragmentadas. En este nivel de criticidad, se requiere un entorno corporativo integral desde el minuto cero. Por esta razón, la arquitectura del cliente web se construirá directamente sobre *Angular*.

- *Estructura empresarial y mantenibilidad:* A diferencia de otras librerías donde el desarrollador debe armar su propia arquitectura, Angular proporciona un marco de trabajo completo fuertemente tipado con TypeScript y un sistema de Inyección de Dependencias nativo. Para un sistema financiero que va a escalar y que será mantenido por distintos equipos a lo largo del tiempo, esta rigidez es una ventaja enorme: obliga a mantener el código ordenado, modular y bajo un mismo estándar, reduciendo drásticamente la deuda técnica.
- *Mitigación nativa de vulnerabilidades web:* En lugar de depender de múltiples paquetes de terceros para asegurar la aplicación, este framework ya incluye escudos integrados contra las amenazas más críticas (alineado a los estándares de OWASP). Su sanitización automática bloquea por defecto los intentos de inyección de código (Cross-Site Scripting - XSS) y cuenta con mecanismos directos para defender la sesión del usuario contra la falsificación de peticiones (CSRF), una característica innegociable al manejar credenciales y fondos bancarios.

2.3. Backend

El verdadero reto de esta plataforma no está en las pantallas, sino en la orquestación de los datos. El ejercicio nos exige integrar la información de una plataforma Core con un sistema independiente, procesar transferencias y exponer todo de forma segura mediante un API Gateway sin que el usuario perciba demoras. Para manejar este nivel de estrés transaccional y construir nuestros microservicios, el motor central será *.NET (C#)*.

- *Precisión absoluta y rendimiento asíncrono:* Cuando hablamos de dinero, un error de redondeo por usar el tipo de dato incorrecto es catastrófico. C# nos otorga un control estricto sobre los tipos de datos (destacando su tipo decimal nativo, diseñado específicamente para cálculos financieros exactos). Sumado a esto, .NET Core corriendo en contenedores nos permite manejar miles de llamadas asíncronas por segundo, lo cual es vital para que el API Gateway pueda consultar saldos y movimientos sin saturarse.
- *Madurez en seguridad e integraciones corporativas:* El requerimiento nos obliga a implementar flujos de OAuth 2.0 y conectarnos a por lo menos dos sistemas externos para el envío de notificaciones. El ecosistema de .NET brilla en este escenario: cuenta con librerías nativas y respaldadas por la industria para manejar tokens JWT, encriptación de payloads y delegación de identidades. Esto nos evita reinventar la rueda o depender de paquetes de terceros de dudosa procedencia para asegurar las transacciones.

2.4. Infraestructura en la Nube

He seleccionado *Microsoft Azure* como el proveedor principal en la nube para desplegar toda la solución de la entidad BP. Tomé esta decisión basándome en las garantías que nos exige el caso de estudio: existe presupuesto asignado para asegurar alta disponibilidad y recuperación ante desastres, y las regulaciones financieras son no negociables. Azure nos resuelve esta carga operativa desde la base por lo siguiente:

- *Cumplimiento normativo certificado de fábrica:* En lugar de construir desde cero los controles de seguridad exigidos por las entidades de control bancario, Azure ya cuenta con el portafolio de certificaciones más robusto del mercado (incluyendo PCI-DSS para el manejo de tarjetas e ISO 27001). Esto agiliza enormemente las auditorías legales a las que el banco deberá someterse y reduce drásticamente el riesgo de multas.

- *Ecosistema nativo para orquestación y backend:* Al tener toda nuestra capa de microservicios construida en C# (.NET), Azure es el entorno más natural y eficiente para su despliegue. Nos permite integrar servicios administrados que son vitales para esta arquitectura, como *Azure API Management* para controlar el tráfico del API Gateway, y *Azure Cosmos DB* para consultar la base de datos independiente con latencias de milisegundos. Además, el uso de *Azure Application Insights* nos garantiza un monitoreo profundo en tiempo real.

3. ARQUITECTURA DE AUTENTICACIÓN Y SEGURIDAD

El control de acceso es el perímetro más crítico de la entidad BP. El diseño no puede depender de flujos de autenticación obsoletos ni almacenar datos biométricos de manera insegura. Toda la delegación de identidad y el Onboarding estarán centralizados para garantizar fricción cero al usuario y máxima seguridad al banco.

3.1. Flujo Óptimo para OAuth 2.0

Para proteger los canales digitales de los clientes, abarcando tanto la plataforma web como la aplicación móvil, queda completamente descartado el uso de flujos heredados como el Implicit Flow. Hoy en día, esta práctica representa una vulnerabilidad inaceptable en el sector financiero. El estándar definitivo que implementaremos es el Authorization Code Flow con la extensión de seguridad PKCE.

- *Protección dinámica de credenciales:* Al tratarse de aplicaciones públicas donde es imposible ocultar credenciales estáticas de forma segura, ya que el código web siempre expone información en el navegador y los binarios móviles son susceptibles a ingeniería inversa, PKCE resuelve este problema generando una clave criptográfica única en cada intento de inicio de sesión. Esto garantiza que, incluso si un atacante logra interceptar el tráfico de red, le resulte matemáticamente imposible canjear el código de autorización por un token de acceso real, blindando por completo las cuentas de los usuarios.
- *Integración natural con la infraestructura actual:* Considerando que la entidad BP ya dispone de una solución interna para gestionar OAuth 2.0, aplicar este flujo

con PKCE resulta ideal al ser el estándar principal soportado por cualquier proveedor de identidad moderno. Nuestra capa de presentación web y móvil tendrá la única responsabilidad de iniciar el desafío criptográfico, delegando toda la carga de validación pesada directamente a este servicio central.

3.2. Integración de Onboarding Biométrico y Accesos Posteriores

El diseño del Onboarding móvil debe integrarse directamente al ciclo de vida del usuario sin comprometer la base de datos de BP con un almacenamiento riesgoso de imágenes faciales. Para orquestar este proceso, nos apoyaremos en el producto de identidad del banco integrando el motor de Azure AI Face para el análisis cognitivo y delegaremos el acceso continuo al estándar FIDO2 junto con WebAuthn.

- *Onboarding con validación de prueba de vida:* Durante el registro del nuevo cliente, la aplicación móvil capturará el rostro y enviará la solicitud a través del API Gateway hacia el servicio de inteligencia artificial. Esta herramienta de nivel industrial no solo compara las facciones contra el documento de identidad, sino que aplica validación de vitalidad para evitar que se utilicen fotos impresas o videos para engañar al sistema. Una vez que el motor aprueba la identidad, el sistema autoriza la creación del perfil y emite el token inicial de manera segura.
- *Acceso continuo descentralizado:* Una vez enrolado el usuario, enviar su rostro al servidor en cada inicio de sesión resulta ineficiente y muy riesgoso. Para los accesos diarios orientados a transferencias rápidas o consultas, la arquitectura delega la validación de identidad directamente al hardware criptográfico del dispositivo móvil, aprovechando los sensores nativos de reconocimiento facial o dactilar. Utilizando el protocolo WebAuthn, el teléfono firma un desafío criptográfico de manera local sin que los datos biométricos viajen jamás por la red. Esto hace que el inicio de sesión sea instantáneo y totalmente resistente a ataques de suplantación o phishing.

4. CAPA DE INTEGRACIÓN, MICROSERVICIOS Y PATRONES DE DATOS

El corazón transaccional del sistema requiere orquestar múltiples fuentes de información sin penalizar el tiempo de respuesta hacia el cliente. Para lograr esto de forma segura, la arquitectura implementa una pasarela de APIs centralizada y aplica patrones de diseño de software específicos orientados a resolver la persistencia de alta velocidad y el registro de auditoría estricto.

4.1. API Gateway y Expansión de Microservicios

La capa de integración estará gobernada por un componente centralizado como Azure API Management. Este elemento actuará como el único punto de entrada expuesto a internet, enrutando las peticiones validadas hacia los tres microservicios principales exigidos por el negocio que son el servicio de datos básicos, el de movimientos y el motor de transferencias.

Para mejorar significativamente el rendimiento de la arquitectura y evitar latencias al consultar el sistema Core bancario y la plataforma independiente de forma separada, propongo agregar un cuarto componente denominado Servicio Agregador de Cliente. Este microservicio aplicará el patrón de diseño API Composition, encargándose de consultar ambos sistemas backend en paralelo y unificar la información en una sola respuesta optimizada lista para ser consumida por las aplicaciones frontend.

4.2. Persistencia para Clientes Frecuentes mediante Cache-Aside

Acceder repetidamente al sistema Core bancario para consultar saldos o movimientos recientes de clientes muy activos es ineficiente y puede saturar la infraestructura heredada. Para resolver este requerimiento puntual, el diseño incorpora el patrón arquitectónico Cache-Aside soportado por un motor de datos en memoria ultrarrápida como Redis.

Bajo este patrón, cuando un cliente frecuente ingresa a la aplicación, el microservicio consulta primero esta capa de memoria veloz. Si la información actualizada se encuentra allí, la devuelve en milisegundos. En caso contrario, el servicio acude al sistema Core, responde al cliente y simultáneamente guarda una copia temporal en la

memoria rápida con un tiempo de expiración prudente. Esta estrategia absorbe la mayor parte del tráfico pesado y protege los sistemas críticos del banco.

4.3. Auditoría Desacoplada con Patrón Publish-Subscribe

El sistema exige registrar absolutamente todas las acciones del usuario, pero escribir en una base de datos de auditoría en medio de una transferencia bancaria suma una latencia inaceptable al proceso principal. Para evitar este cuello de botella, implementaremos el patrón de diseño Publish-Subscribe utilizando un bus de eventos corporativo.

Cada vez que un microservicio ejecuta una transacción exitosa, emite un evento ligero hacia el bus de mensajes y da por terminada su tarea de cara al usuario de forma inmediata. Paralelamente, un microservicio dedicado exclusivamente a la auditoría consume estos eventos en segundo plano y los inserta en una base de datos no relacional de alta escritura, garantizando un registro inmutable y estricto sin afectar la percepción de velocidad en la aplicación móvil o web.

4.4. Sistema Desacoplado de Notificaciones Omnicanal

La normativa bancaria exige notificar los movimientos realizados mediante al menos dos sistemas. Para mantener la cohesión del diseño, la emisión de estas alertas también se conectará al bus de eventos mencionado en el punto anterior.

Se construirá un microservicio de Notificaciones encargado de escuchar las transferencias aprobadas y despachar los avisos a través de dos proveedores externos probados en la industria. El primer canal utilizará plataformas de mensajería en la nube para emitir notificaciones Push directas y seguras hacia la aplicación móvil. El segundo canal se integrará con proveedores de comunicaciones corporativas para el envío de mensajes de texto cortos y correos electrónicos transaccionales que incluyan los comprobantes digitales.

5. ARQUITECTURA DE SOFTWARE (MODELO C4)

Para visualizar la solución propuesta de manera estructurada y comprensible para todos los niveles técnicos y gerenciales del banco, se ha adoptado el estándar de arquitectura C4. A continuación, se presentan los tres niveles arquitectónicos principales que describen desde la interacción general del usuario hasta el diseño interno del código de los microservicios.

5.1. Nivel 1: Diagrama de Contexto del Sistema

Este nivel ilustra el panorama general. Muestra cómo el Cliente BP interactúa de forma directa con el nuevo Sistema de Banca por Internet y define claramente las fronteras del proyecto al mapear las dependencias con los sistemas externos (Plataforma Core, Proveedor de Identidad y Sistemas de Notificaciones).

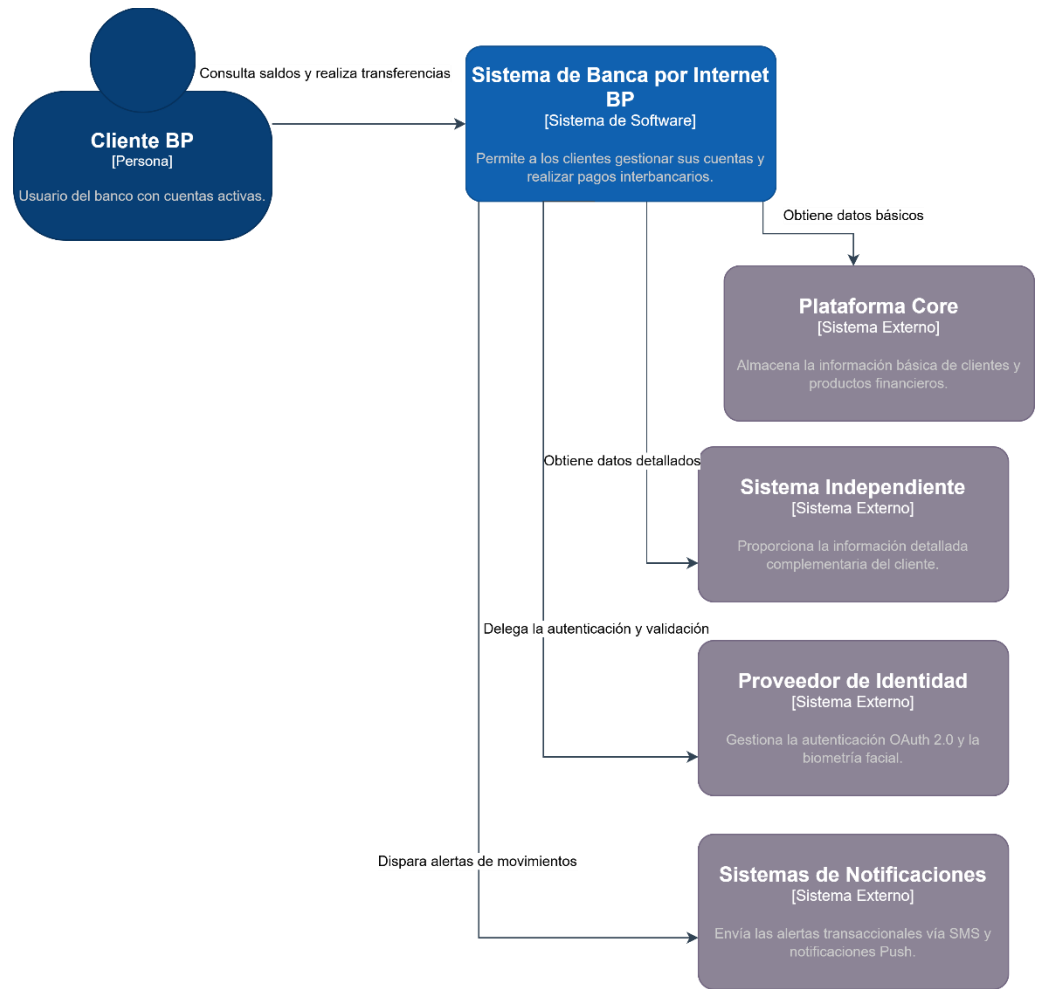


Ilustración 1: Diagrama de Contexto: Interacciones del Sistema de Banca.

5.2. Nivel 2: Diagrama de Contenedores

Este nivel desglosa la estructura interna del Sistema de Banca por Internet, detallando las piezas tecnológicas que lo conforman y sus interacciones. El diagrama

expone la clara separación de responsabilidades entre las interfaces de usuario (Web SPA y Móvil), la barrera perimetral de seguridad y enrutamiento (API Gateway), y el núcleo transaccional del backend. Este último opera bajo una arquitectura orquestada mediante microservicios independientes, respaldada por almacenamiento optimizado en memoria y mensajería asíncrona a través de buses de eventos.

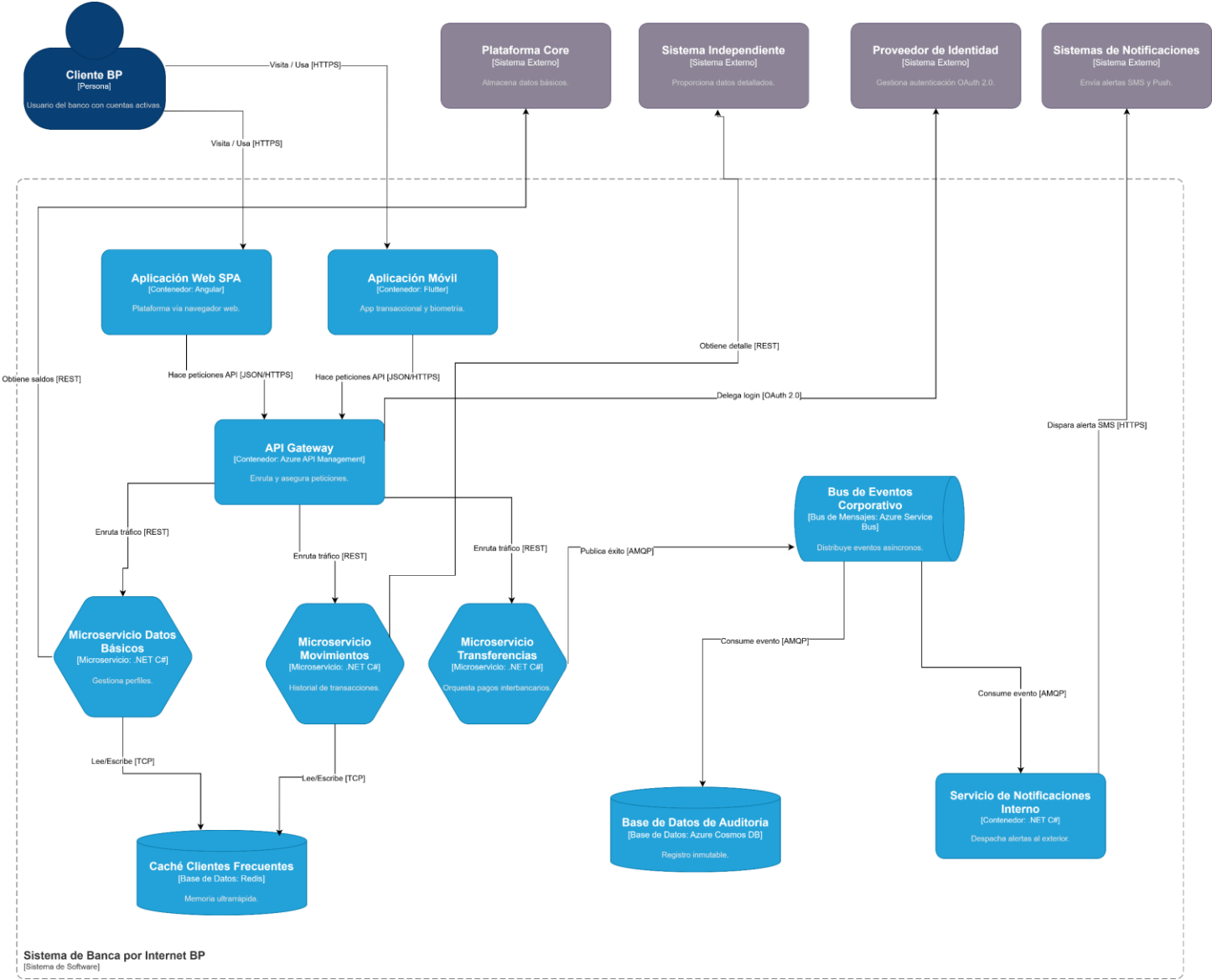


Ilustración 2 : Diagrama de Contenedores: Arquitectura y tecnologías implementadas.

5.3. Nivel 3: Diagrama de Componentes (Microservicio de Transferencias)

Este diagrama proporciona una vista detallada de la arquitectura interna del Microservicio de Transferencias, el cual constituye el motor transaccional crítico de la

plataforma. En esta representación se define la topología del software y el flujo de ejecución a través de sus componentes especializados, abarcando desde los controladores REST y los *middlewares* de validación JWT, hasta los gestores de la lógica de negocio y los clientes de integración HTTP.

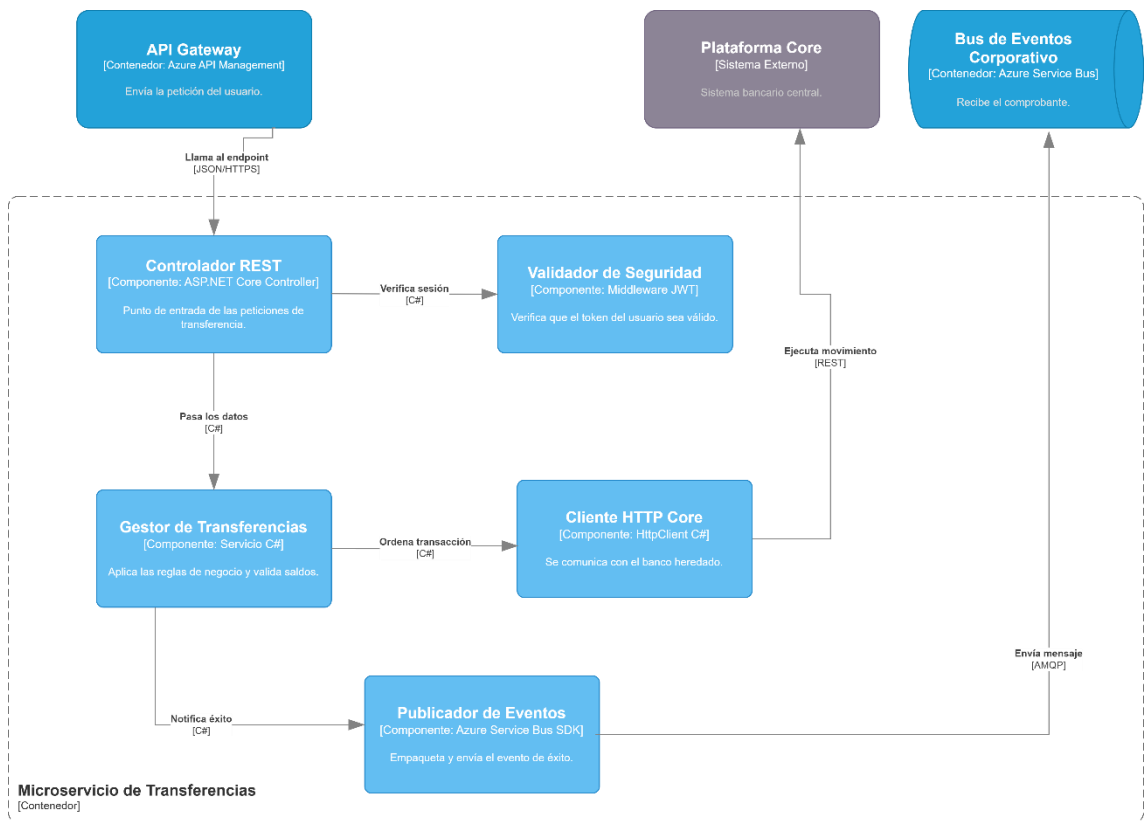


Ilustración 3: Diagrama de Componentes: Estructura del Microservicio de Transferencias.

6. CONSIDERACIONES NO FUNCIONALES Y NORMATIVAS

El diseño de la arquitectura propuesta no solo resuelve los flujos transaccionales del Sistema de Banca por Internet, sino que establece una base tecnológica robusta para garantizar la continuidad operativa y la protección de los activos digitales. A continuación, se detallan los pilares no funcionales que rigen esta implementación.

6.1. Seguridad y Cumplimiento Bancario

Dada la naturaleza crítica de las operaciones financieras, la plataforma se estructura bajo un enfoque de seguridad en capas y un modelo de "confianza cero". El

acceso a los recursos se gestiona de manera centralizada delegando la validación inicial a un Proveedor de Identidad externo que opera bajo el estándar OAuth 2.0. En esta arquitectura, el API Gateway asume el rol de barrera perimetral estricta, encargándose de interceptar y validar los tokens JWT de cada petición antes de permitir que el tráfico alcance la red interna donde residen los microservicios.

Por otro lado, para dar estricto cumplimiento a las normativas de la superintendencia de bancos y entidades regulatorias, se ha integrado un mecanismo robusto de trazabilidad y auditoría. Cada vez que el sistema procesa una transacción exitosa, se publica un evento que queda registrado permanentemente en una base de datos documental, en este caso Azure Cosmos DB. Esta estrategia genera un registro de auditoría completamente inmutable y transparente, el cual resulta vital para respaldar futuras inspecciones legales, auditorías internas o conciliaciones contables del banco.

6.2. Alta Disponibilidad y Tolerancia a Fallos

Para asegurar que los clientes mantengan un acceso continuo a sus fondos y servicios las 24 horas del día, el diseño distribuido de la plataforma se enfoca en mitigar activamente cualquier punto único de fallo. Esto se logra en gran medida mediante el desacoplamiento asíncrono que proporciona la implementación de un bus de eventos corporativo, como Azure Service Bus. De esta manera, si los sistemas secundarios de notificaciones o las bases de datos de auditoría sufren caídas temporales, el flujo principal de las transferencias no se bloquee, ya que los mensajes permanecen encolados de forma segura hasta el restablecimiento total del servicio.

Adicionalmente, la arquitectura contempla estrategias avanzadas de resiliencia al momento de integrarse con sistemas heredados o *legacy*, como es el caso de la Plataforma Core del banco. La comunicación con estos componentes externos incluye la implementación de patrones de resiliencia de software, destacando el uso de *Circuit Breaker* y reintentos exponenciales. Estas medidas preventivas son fundamentales para encapsular las fallas y evitar que una latencia inusual o una caída en un sistema externo provoque una cascada de errores que termine afectando a toda nuestra plataforma de microservicios.

6.3. Rendimiento y Escalabilidad

El rendimiento del sistema se optimiza significativamente mediante la incorporación de estrategias de caché, utilizando Redis para almacenar en memoria

ultrarrápida los datos de uso constante. Al mantener las consultas de perfiles de usuario y los historiales de movimientos más frecuentes en esta capa de acceso rápido, se reduce drásticamente el volumen de peticiones repetitivas que se envían hacia el sistema Core. Esta descarga de trabajo en el backend tradicional es lo que nos permite garantizar tiempos de respuesta de apenas unos milisegundos, brindando una experiencia fluida tanto en la aplicación web como en la aplicación móvil.

Finalmente, al encapsular toda esta lógica de negocio en microservicios independientes desarrollados con .NET, la infraestructura adquiere la capacidad nativa de escalar horizontalmente bajo demanda. Esta elasticidad es una ventaja competitiva enorme, ya que, en días de alta concurrencia transaccional, como las quincenas o los cierres de mes, el banco tiene la flexibilidad de inyectar recursos computacionales exclusivamente al Microservicio de Transferencias. Así, se soporta el pico de usuarios sin necesidad de sobredimensionar componentes inactivos, optimizando de manera inteligente los costos de operación en la nube.